

LIBRARY MANAGEMENT SYSTEM

DBMS Project Synopsis

Course Code: UCS310

BTech 2nd Year

Submitted to:

Dr. Shifali Goyal

Thapar Institute of Engineering and Technology

Group Members:

Harsh Runda (1024170151)

Aviral Agrawal (1024170169)

Vaidik Bhardwaj (1024170234)

Academic Year: 2025-2026

1. INTRODUCTION

With the growing demand for organized and digitized resource management, libraries require efficient systems to catalog books, manage memberships, track borrowings, and handle fines at scale. A Library Management System (LMS) serves as the central platform for administering all library operations, from book acquisition to member record maintenance.

Currently, many libraries rely on manual registers, spreadsheets, and disconnected tools, leading to data redundancy, delayed processing, inaccurate fine calculations, and poor inventory visibility.

1.1 Motivation

- Need for an efficient, centralized system to manage thousands of books and member records.
- Ensuring transparent tracking of book issues, returns, reservations, and overdue fines.
- Reducing manual errors and improving data integrity in library workflows.
- Enabling real-time availability monitoring for members and librarians.

1.2 Importance of DBMS

A Database Management System (DBMS) is essential for this system because:

- Centralizes data storage for books, members, staff, transactions, and fines.

- Ensures ACID properties (Atomicity, Consistency, Isolation, Durability) for critical library transactions.
- Provides efficient querying and reporting capabilities for inventory and usage analysis.
- Maintains data consistency through referential integrity constraints.

2. PROBLEM STATEMENT

Managing library operations through manual or disconnected systems involves:

- Managing information about large book collections across multiple categories and authors.
- Storing membership data and tracking borrowing privileges for each member.
- Processing book issues, returns, and reservations across multiple branches.
- Tracking overdue items and accurately computing fines.

2.1 Limitations of Manual/File-Based Systems

- Data redundancy and inconsistency across different registers and spreadsheets.
- Difficult to handle large-scale concurrent book and member data access.
- No built-in mechanisms for data validation and borrowing limit constraints.
- Slow query processing for complex reporting and inventory analytics.
- No recovery mechanism in case of system failure or data loss.

2.2 Scope of Database Solution

The proposed DBMS solution will:

- Store and manage information about books, members, staff, and transactions.
- Support book reservations, issues, and returns with proper due-date tracking.
- Ensure data integrity and consistency throughout all library operations.

3. OBJECTIVES OF THE PROJECT

- To design a normalized database schema using E-R data modelling for the library management system.
- To convert the ER model into relational tables with proper keys and relationships.
- To apply normalization techniques (1NF, 2NF, 3NF/BCNF) to eliminate data anomalies.
- To implement the database schema using SQL DDL and DML commands.

- To develop PL/SQL constructs including stored procedures, functions, triggers, and cursors for automated operations.
- To ensure data consistency and integrity using constraints, transactions, and exception handling.

4. SCOPE OF THE PROJECT

4.1 Functional Boundaries

The system handles:

- Book Catalog and Inventory Management
- Member Registration and Profile Management
- Book Issue, Return, and Reservation Tracking
- Fine Calculation and Payment Processing
- Staff and Role Management

4.2 Types of Users

- **Admin:** Manages the entire system, users, book catalog, and configuration.
- **Librarian:** Issues and receives books, manages reservations, and processes fines.
- **Member:** Searches the catalog, reserves books, and views borrowing history.

4.3 Modules

- Book Catalog & Inventory Module
- Member Management Module
- Book Issue & Return Module
- Reservation Management Module
- Fine & Payment Module

5. PROPOSED SYSTEM DESCRIPTION

5.1 Overall Architecture

The Library Management System is a centralized database solution that manages the complete library lifecycle. It integrates book records, member profiles, staff information, transaction logs, reservation queues, and fine records in a single normalized database.

5.2 Key Features

- **Centralized Database:** Single source of truth for all library and catalog data.
- **Multi-Copy Book Support:** Handle multiple copies of the same title with individual tracking.

- **Data Integrity:** Constraints and triggers ensure no duplicate issues or invalid returns.
- **Real-time Updates:** Automated availability and fine tracking for members.
- **Transaction Management:** ACID-compliant issue, return, and payment operations.

6. DATABASE DESIGN

6.1 Entity-Relationship Diagram

Entity	Description
BOOK	Stores title, ISBN, genre, publisher, and edition details
AUTHOR	Author name and biographical information
MEMBER	Member personal details, membership type, and join date
STAFF	Librarian and admin staff profiles and roles
BOOK_COPY	Individual physical copies of each book with availability status
ISSUE	Records of book copies issued to members with due dates
RESERVATION	Member reservations for currently unavailable books
FINE	Overdue fines associated with each late return transaction

6.2 Relationships and Cardinality

Relationship	Cardinality	Description
AUTHOR → BOOK	M : M	Many authors can write many books
BOOK → BOOK_COPY	1 : M	One book title has many physical copies
MEMBER → ISSUE	1 : M	One member can have multiple active issues
BOOK_COPY → ISSUE	1 : M	One copy can be issued multiple times (sequentially)
MEMBER → RESERVATION	1 : M	One member can reserve multiple books
BOOK → RESERVATION	1 : M	One book can be reserved by multiple members
ISSUE → FINE	1 : 1	Each overdue issue generates one fine record

STAFF → ISSUE	1 : M	One staff member processes many issue transactions
---------------	-------	--

7. RELATIONAL SCHEMA

BOOK (Book_ID, ISBN, Title, Genre, Publisher, Edition, Year_Published)

AUTHOR (Author_ID, Name, Nationality, Bio)

BOOK_AUTHOR (Book_ID, Author_ID) — Junction table for M:M relationship

MEMBER (Member_ID, Name, Email, Mobile_No, Date_of_Birth, Membership_Type, Join_Date, Status)

STAFF (Staff_ID, Name, Email, Role, Mobile_No, Join_Date)

BOOK_COPY (Copy_ID, Book_ID, Condition, Availability_Status, Location_Shelf)

ISSUE (Issue_ID, Copy_ID, Member_ID, Staff_ID, Issue_Date, Due_Date, Return_Date, Status)

RESERVATION (Reservation_ID, Book_ID, Member_ID, Reservation_Date, Expiry_Date, Status)

FINE (Fine_ID, Issue_ID, Amount, Fine_Date, Payment_Status, Payment_Date)

8. NORMALIZATION

8.1 Functional Dependencies

- **BOOK:** Book_ID → {ISBN, Title, Genre, Publisher, Edition, Year_Published}
- **MEMBER:** Member_ID → {Name, Email, Mobile_No, Date_of_Birth, Membership_Type, Join_Date}
- **BOOK_COPY:** Copy_ID → {Book_ID, Condition, Availability_Status, Location_Shelf}
- **ISSUE:** Issue_ID → {Copy_ID, Member_ID, Staff_ID, Issue_Date, Due_Date, Return_Date}
- **FINE:** Fine_ID → {Issue_ID, Amount, Fine_Date, Payment_Status}

8.2 Normalization Process

First Normal Form (1NF): All attributes contain atomic values with no repeating groups. All tables meet 1NF requirements.

Second Normal Form (2NF): All non-key attributes are fully dependent on the primary key. No partial dependencies exist in any composite key table (e.g., BOOK_AUTHOR only stores the relationship).

Third Normal Form (3NF): No transitive dependencies exist. All non-key attributes depend only on the primary key, not on other non-key attributes. The final schema is in 3NF.

9. DATABASE IMPLEMENTATION

9.1 SQL DDL Commands

- CREATE commands: CREATE TABLE for all nine entities with constraints
- ALTER commands: Modify table structures and add/drop constraints
- DROP commands: Remove tables and indexes when needed

9.2 SQL DML Commands

- INSERT: Add new books, members, staff, copies, and fine records
- UPDATE: Modify book details, update copy availability and fine payment status
- DELETE: Remove expired reservations or deactivated member accounts

9.3 SQL SELECT Queries

- Joins: Inner and outer joins to link books with copies, issues, and members
- Subqueries: Find most borrowed books, members with outstanding fines
- Aggregate Functions: COUNT (copies), SUM (fine amounts), AVG (borrowing duration)
- GROUP BY & HAVING: Genre-wise inventory, overdue rates per membership type
- Views: Create virtual tables for available books and member borrowing dashboards

10. PL/SQL COMPONENTS

10.1 Stored Procedures

- IssueBook(): Automate book issue with eligibility and availability checks
- ReturnBook(): Process book returns and trigger fine calculation if overdue
- GenerateMemberReport(): Produce comprehensive borrowing history and fine summaries

10.2 Functions

- CalculateFine(): Return overdue fine amount based on days late and membership type
- CheckAvailability(): Verify if at least one copy of a book is currently available
- GetBorrowingCount(): Return the number of active issues for a given member

10.3 Triggers

- **AfterIssueInsert:** Update BOOK_COPY availability status to 'Issued' automatically
- **AfterReturnUpdate:** Reset BOOK_COPY status to 'Available' and notify reservation queue
- **AfterFinePayment:** Mark fine as paid and update member account standing

10.4 Cursors

- **OverdueCursor:** Iterate through all overdue issues for batch fine generation
- **ReservationCursor:** Process the reservation queue when a book copy becomes available

10.5 Exception Handling

- Handle constraint violations (primary key, foreign key, duplicate reservations)
- Catch business logic errors (borrow limit exceeded, expired membership)
- Log errors and rollback on critical transaction failures

11. TRANSACTION MANAGEMENT & CONCURRENCY

11.1 ACID Properties

- **Atomicity:** All-or-nothing book issue and fine payment transactions
- **Consistency:** Database constraints always satisfied during any operation
- **Isolation:** Independent concurrent member and librarian transactions
- **Durability:** Committed issue and payment records are permanent

11.2 Transaction Control

- **COMMIT:** Finalize successful issue, return, and payment entries
- **ROLLBACK:** Undo failed reservation or fine processing attempts
- **SAVEPOINT:** Create recovery points within multi-step return transactions

11.3 Concurrency Control

- Row-level locking (SELECT ... FOR UPDATE) to prevent simultaneous issue of the same book copy
- Read locks for catalog queries, write locks for issue and return operations

12. TOOLS & TECHNOLOGIES USED

Component	Technology
DBMS	PostgreSQL 15 (via Supabase)
Query Language	SQL (Structured Query Language)

Procedural Language	PL/pgSQL (PostgreSQL Procedural Language)
Backend Framework	Next.js 14 (App Router, TypeScript)
Frontend UI	React, Tailwind CSS, shadcn/ui, Recharts
Database Client	Supabase JavaScript SDK (@supabase/ssr)
IDE / Tool	Supabase SQL Editor, VS Code

13. ACTUAL OUTCOMES

The following outcomes were achieved through the implementation of this project:

- Implemented a 9-table normalized PostgreSQL schema in Supabase (3NF), with UUID primary keys, CHECK constraints as ENUMs, and a partial unique index on RESERVATION for duplicate prevention.
- Created 5 SQL views: `available_books_view`, `member_borrowing_dashboard`, `overdue_issues_view`, `library_stats_view`, and `top_borrowed_books` — demonstrating multi-table JOINS, `STRING_AGG`, `COUNT` with `FILTER`, `DATE` arithmetic, and aggregate functions.
- Implemented 7 PL/pgSQL stored functions: `issue_book()`, `return_book()`, `calculate_fine()`, `check_availability()`, `get_borrowing_count()`, `process_reservation_queue()`, and `generate_member_report()` — callable via Supabase RPC.
- Implemented 3 PostgreSQL triggers: `trg_after_issue_insert` (auto-sets book copy to Issued), `trg_after_return_update` (resets availability and processes reservation queue via cursor), and `trg_after_fine_payment` (reactivates suspended members when all fines cleared).
- ACID-compliant issue and return transactions: `issue_book()` uses `SELECT ... FOR UPDATE` row-level locking to prevent concurrent duplicate issues, and `RAISE EXCEPTION` triggers automatic `ROLLBACK` on constraint violations.
- Built a full-stack Next.js 14 application with 7 modules: Dashboard, Books Catalog, Member Management, Issue/Return, Reservation Queue, Fines Payment, and Analytics/Reports.
- Implemented an Analytics/Reports page with a **SQL Transparency** feature: each chart includes a collapsible panel showing the exact SQL query, view definition, or PL/pgSQL function used — directly demonstrating DBMS concepts to evaluators.

- Visualized borrowing trends (line chart via DATE_TRUNC week-grouping), genre distribution (bar chart via GROUP BY), and top-borrowed books (JOIN + aggregate view) using Recharts.
- Implemented role-based UI (Admin / Librarian / Member) via a mock role-switcher, controlling visibility of management features and action buttons.
- Updated technology stack from MySQL 8.0 + MySQL Workbench to **PostgreSQL (Supabase) + PL/pgSQL + Next.js 14 (TypeScript)**, retaining all DBMS concepts and adding real-time web interface.
- Seeded 30 book copies across 15 titles and 16 authors (including multi-author CLRS demonstrating BOOK_AUTHOR M:M junction), 10 members, 3 staff, 33 issues (25 historical + 8 active), 5 reservations, and 8 fines for complete demo data.

14. CONCLUSION

The Library Management System project demonstrates the practical application of database design principles, normalization techniques, and PL/SQL programming. By implementing this system, we showcase:

- The importance of proper database design for managing complex library data scenarios.
- How to ensure data consistency and integrity in mission-critical library applications.
- Advanced SQL and PL/SQL techniques for automating library business processes.
- Best practices for transaction management and concurrency control in multi-user environments.

This project provides a solid foundation for understanding database management systems and their applications in large-scale library and resource administration systems. The implementation was extended beyond the original synopsis to include a fully functional web application interface built with Next.js 14 and Supabase, making all DBMS concepts observable through a real-time interactive dashboard.